MLOps, Deployment & Cloud

Take models from notebook to reliable production systems.

Contents

#	Topic	Level
1	ML system lifecycle and MLOps maturity	Beginner
2	Project structure, configs and reproducibility	Beginner
3	Experiment tracking & model registry (MLflow, W&B)	Intermediate
4	Data & model versioning (DVC, Git LFS, lakeFS)	Intermediate
5	Serving models with FastAPI / Flask	Beginner
6	Docker and containers	Intermediate
7	Kubernetes, scaling and serverless basics	Advanced
8	CI/CD with GitHub Actions	Intermediate
9	Workflow orchestration: Airflow, Prefect, Kubeflow, Dagster	Advanced
10	Feature stores and data pipelines (Feast, dbt, Kafka, Spark)	Advanced
11	Cloud platforms: AWS SageMaker, GCP Vertex AI, Azure ML, Hugging Face Spaces	Intermediate
12	Monitoring: drift, data quality, performance, logging	Advanced
13	Testing ML systems and A/B tests in production	Advanced
14	Security, privacy, fairness and governance	Advanced
15	Edge & mobile deployment: TFLite, CoreML, ONNX, WebAssembly	Advanced
16	LLMOps: prompt versioning, tracing, cost & latency control	Advanced

1. ML system lifecycle and MLOps maturity [Beginner]

Explanation (English)

Data -> training -> validation -> deployment -> monitoring -> retraining. MLOps applies DevOps practices (automation, versioning, CI/CD) to ML.

Tanglish (Tamil + English) Vilakkam

MLOps = ML + DevOps. Automation, versioning, monitoring.

Learn more

YouTube search: mlops explained

2. Project structure, configs and reproducibility [Beginner]

Explanation (English)

Use src/ layout, config files (YAML/Hydra), requirements/poetry/uv lock files, fixed seeds, Makefile.

Tanglish (Tamil + English) Vilakkam

Clean project structure, config files, requirements.txt.

Example / Program

```
proj/
  data/  notebooks/  src/  tests/  configs/  requirements.txt  README.md
```

Learn more

YouTube search: Project structure, configs and reproducibility tutorial

3. Experiment tracking & model registry (MLflow, W&B) [Intermediate]

Explanation (English)

Log params, metrics, artefacts, code version; compare runs; promote models through staging to production in a registry.

Tanglish (Tamil + English) Vilakkam

MLflow-la experiments track pannunga.

Example / Program

```
import mlflow
mlflow.set_experiment('demo')
with mlflow.start_run():
    mlflow.log_param('C',1);mlflow.log_metric('f1',.9)
```

Learn more

YouTube search: Experiment tracking & model registry (MLflow, W&B) tutorial

4. Data & model versioning (DVC, Git LFS, lakeFS) [Intermediate]

Explanation (English)

Version large datasets and models alongside code for exact reproducibility.

Tanglish (Tamil + English) Vilakkam

DVC data version pannum.

Example / Program

```
dvc init
dvc add data/train.csv
git add data/train.csv.dvc .gitignore
git commit -m 'track data'
```

Learn more

YouTube search: Data & model versioning (DVC, Git LFS, lakeFS) tutorial

5. Serving models with FastAPI / Flask [Beginner]

Explanation (English)

Load model once at start-up; validate input with Pydantic; return predictions as JSON; add health endpoint.

Tanglish (Tamil + English) Vilakkam

FastAPI-la model serve.

Example / Program

```
from fastapi import FastAPI
from pydantic import BaseModel
import joblib
app=FastAPI();m=joblib.load('m.joblib')
class In(BaseModel): x:list[float]
@app.post('/predict')
def p(i:In): return {'y':int(m.predict([i.x])[0])}
```

Learn more

YouTube search: Serving models with FastAPI / Flask tutorial

6. Docker and containers [Intermediate]

Explanation (English)

Package code + dependencies into an image so it runs identically anywhere. Write small, cached, non-root images.

Tanglish (Tamil + English) Vilakkam

Docker - everywhere same-a run aagum.

Example / Program

```
# Dockerfile
FROM python:3.11-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .
CMD ["uvicorn","app:app","--host","0.0.0.0","--port","8000"]
```

Learn more

YouTube search: Docker and containers tutorial

7. Kubernetes, scaling and serverless basics [Advanced]

Explanation (English)

Deploy containers with Deployments/Services, autoscale, GPU nodes; serverless (Lambda, Cloud Run) for light workloads.

Tanglish (Tamil + English) Vilakkam

Kubernetes scale panna.

Learn more

YouTube search: kubernetes for machine learning

8. CI/CD with GitHub Actions [Intermediate]

Explanation (English)

Run linters, tests and image builds on each push; deploy automatically after checks pass.

Tanglish (Tamil + English) Vilakkam

GitHub Actions - auto test & deploy.

Example / Program

```
# .github/workflows/ci.yml
on: [push]
jobs:
  t:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with: {python-version: '3.11'}
      - run: pip install -r requirements.txt pytest && pytest
```

Learn more
YouTube search: CI/CD with GitHub Actions tutorial

9. Workflow orchestration: Airflow, Prefect, Kubeflow, Dagster [Advanced]

Explanation (English)
Schedule and monitor data/training pipelines as DAGs.

Tanglish (Tamil + English) Vilakkam
Pipelines schedule panna Airflow/Prefect.

Learn more
YouTube search: apache airflow tutorial

10. Feature stores and data pipelines (Feast, dbt, Kafka, Spark) [Advanced]

Explanation (English)
Share consistent features between training and serving; batch vs streaming pipelines.

Tanglish (Tamil + English) Vilakkam
Feature store - training/serving same features.

Learn more
YouTube search: feature store feast explained

11. Cloud platforms: AWS SageMaker, GCP Vertex AI, Azure ML, Hugging Face Spaces [Intermediate]

Explanation (English)
Managed training, hosting, pipelines, GPUs. Learn IAM, storage (S3/GCS), compute, cost control.

Tanglish (Tamil + English) Vilakkam
Cloud basics: S3, EC2, SageMaker, Vertex AI.

Learn more
YouTube search: aws sagemaker tutorial

12. Monitoring: drift, data quality, performance, logging [Advanced]

Explanation (English)
Track input/feature drift, prediction drift, latency, errors and business KPIs; alerting and retraining triggers (Evidently, Prometheus, Grafana).

Tanglish (Tamil + English) Vilakkam
Model production-la drift aaguthaa-nu monitor.

Example / Program
```
from evidently.report import Report
from evidently.metric_preset import DataDriftPreset
# Report([DataDriftPreset()]).run(reference_data=a,current_data=b)
```

Learn more
YouTube search: Monitoring: drift, data quality, performance, logging tutorial

13. Testing ML systems and A/B tests in production [Advanced]

Explanation (English)
Unit tests for transforms, data validation tests, model behaviour tests, shadow deployment, canary and online A/B experiments.

Tanglish (Tamil + English) Vilakkam

Shadow, canary, A/B test.

Learn more
YouTube search: testing machine learning systems

14. Security, privacy, fairness and governance [Advanced]

Explanation (English)
Secrets management, PII handling, differential privacy, federated learning, bias audits, model cards, audit logs.

Tanglish (Tamil + English) Vilakkam
Security, privacy, fairness.

Learn more
YouTube search: responsible ai fairness bias audit

15. Edge & mobile deployment: TFLite, CoreML, ONNX, WebAssembly [Advanced]

Explanation (English)
Quantise and convert models for phones, browsers and microcontrollers (TinyML).

Tanglish (Tamil + English) Vilakkam
Mobile / edge deploy.

Learn more
YouTube search: tinyml tflite deployment

16. LLMOps: prompt versioning, tracing, cost & latency control [Advanced]

Explanation (English)
Trace calls (Langfuse, LangSmith), cache responses, batch, route to cheaper models, guardrails, eval in CI.

Tanglish (Tamil + English) Vilakkam
LLMOps - prompts, traces, cost control.

Learn more
YouTube search: llmops langfuse tracing